

07 - Chaînes de caractères

Lors du cours n°2, nous avons parlé des types des variables en C, je fais surtout référence au type `char` qui représente un caractère [02 - Premiers pas > Le type caractère](#).

 Rappel

Le type `char` permet de stocker des caractères, et puisqu'ils sont stockés dans l'ordinateur sous forme d'entier, on peut alors aussi utiliser `char` pour stocker un entier.

Un problème se pose alors...

Une variable de type `char` ne peut stocker qu'un seul et unique caractère, comment peut-on faire pour stocker des mots entiers ? Voir des phrases entières ?

On cherche un moyen de stocker plusieurs lettre dans un même objet. Justement, on sait déjà le faire, par le biais des **tableaux**.

Il y a toujours un problème, une chaîne de caractères est plus qu'un tableau, elle doit être manipulable directement, hors ceci n'est possible que si on connaît sa taille.

Déclaration

Ici, rien ne change vraiment de déclaration d'un tableau :

```
char chaine[t]="...";
```

Où

- `t` représente la taille de la chaîne de caractères.
- `"..."` la chaîne de caractère que l'on stocke

Exemple :

On souhaite stocker le mot `informatique` dans une chaîne de caractères nommée `domaine`, on va alors créer un tableau de caractères et stocker le mot.

```
char domaine[] = "informatique";
```

Voici ci dessous, une version schématique simple qui vous permet de comprendre comment `informatique` est stocké.

i	n	f	o	r	m	a	t	i	q	u	e	\0
0x1000	0x1001	0x1002	0x1003	0x1004	0x1005	0x1006	0x1007	0x1008	0x1009	0x100A	0x100B	0x100C

En fait, lors de la création de la variable `domaine` qui contient la chaîne de caractère "informatique", cela va créer un tableau de caractères `char` contenant chaque lettre du mot et le caractère nul (`\0`) qui permet de marquer la fin de la chaîne de caractères.

Ainsi, dans notre exemple un tableau de 13 éléments est créé, il peut être schématisé *grossièrement* de la manière suivante :

```
[ i, n, f, o, r, m, a, t, i, q, u, e, \0]
```

Chaque caractère est stocké dans un **espace mémoire** consécutive (s'incrémente de 1 pour chaque caractère). On l'aperçoit sur notre image au dessus :

Caractère	Adresse mémoire
i	0x1000
n	0x1001
f	0x1002
⋮	⋮
e	0x100B
\0	0x100C

En gros, chaque lettre de type `char` en C est stocké sur 1 bit, et l'adresse mémoire entre deux caractères est simplement incrémenté de 1. On utilise `\0` comme marque de fin de chaîne car sans lui, le programme ne pourrait pas savoir où s'arrête la chaîne de caractères.

Initialisation

Initialisation explicite

L'**initialisation explicite** consiste à définir chaque caractère du tableau et éventuellement le caractère nul (`'\0'`), obligatoire pour une chaîne de caractères.

Exemple :

```
char chaine[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

Initialisation implicite

L'**initialisation implicite** c'est en fait ce qu'on a vu pour la [07 - Chaînes de caractères > Déclaration](#), on utilise un littéral de chaîne entre guillemets :

Exemple :

```
char chaine[] = "Hello";
```

On peut aussi déclarer un tableau à plus d'éléments que ce qu'on initialise, dans ce cas les éléments non initialisés auront des valeurs dites indéterminées (ou nulles si le tableau est déclaré globalement).

Exemple :

```
char chaine[20] = "Salut";
```

Aspect	Initialisation explicite	Initialisation implicite
Définition	Chaque caractère + <code>'\0'</code> manuel	Littéral de chaîne (<code>"..."</code>)
Ajout de <code>'\0'</code>	manuel	Automatique
Taille du tableau	Fixe, définie par l'utilisateur	Déduite automatiquement si non précisée
Facilité	Moins pratique	Plus simple et sûre

Manipulation des chaînes de caractères

Accès aux éléments

Grâce à la première partie de ce cours [Déclaration](#), on sait qu'une chaîne de caractères en C représente simplement un tableau de caractères. Il est alors possible d'accéder aux éléments d'une chaîne de caractères de la même manière que pour des tableaux classiques. Alors, en considérant une chaîne de caractères `chaine` d'une taille $t \in \mathbb{N}^*$, si on souhaite accéder au n -ième élément avec $n \in [1; t]$ alors on utilisera la forme suivante :

```
char elementN = chaine[n-1]
```

Exemple :

En reprenant l'exemple précédant avec le mot `informatique`, si on souhaite avoir accès à la lettre `m` alors on exécutera le code suivant :

```
#include <stdio.h>

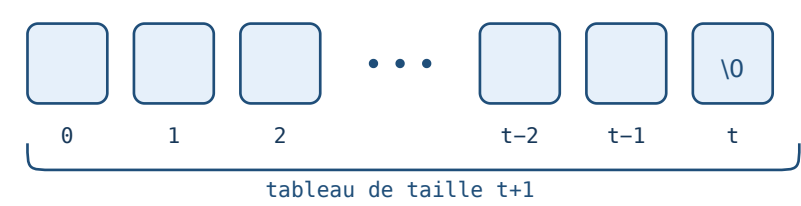
int main() {
    char domaine[] = "informatique";
    char m = domaine[5];
    printf("6e lettre du domaine : %c", m);
    return 0;
}
```

```
6e lettre du domaine : m
```

 **Rappel**

Une chaîne de caractères est un tableau d'éléments de type `char` cela implique que pour y accéder on utilise des entiers appelés indices. Les indices commencent à 0 qui correspond au premier caractère de la chaîne jusqu'à $t - 1$ pour le dernier caractère de la chaîne. Et, le `\0` caractère nul pour marquer la fin de la chaîne est l'élément d'indice t .

Ainsi en considérant une chaîne de t éléments, voici les indices de manière schématique.



On voit donc que pour un mot de taille t , un tableau de taille `t+1` sera créé pour stocker chaque caractère du mot en entier et un emplacement mémoire supplémentaire est ajouté pour stocker le caractère de fin de chaîne `\0`. C'est pour cela que pour un mot de taille t , le tableau qui stocke le mot sera de taille $t + 1$.

Affichage

Mais comment on peut afficher une chaîne de caractères en langage C. On doit vraiment faire comme avec les tableaux classique en parcourant le tableau élément par élément et l'afficher un à un ?

Alors c'est une solution que l'on va regarder, mais ce n'est pas la plus simple ! Il existe plusieurs méthodes pour afficher une chaîne de caractères. Vous le savez déjà mais pour afficher des informations dans la console on utilise `printf` et quand on a besoin d'afficher un certain type de donnée on utilise des **formats** notés `%...`. Et bien sachez qu'il existe un format **réservé aux chaînes de caractères** ! Plutôt pratique non ? C'est le format `%s`.

Exemple :

```
#include <stdio.h>

int main(){
    char chaine[] = "j'écris un truc pour le kiff";

    // Affichage en utilisant le format %s
    printf("%s\n", chaine);
    return 0;
}
```

```
j'écrit un truc pour le kiff
```

C'est la solution la plus simple pour afficher une chaîne de caractère.

On peut aussi faire ce que l'on a vu pendant le cours 6 sur les tableaux, on parcourt chaque élément du tableau grâce à une boucle `for` puis on affiche chaque caractère un à un.

Warning

Dans ces cas là, on a besoin de la taille du tableau pour pouvoir savoir au bout de combien d'itérations on arrête.

On utilise alors une méthode du package `<string.h>`, la méthode `strlen(chaine)` qui permet de donner la taille de la chaîne de caractères sans inclure le marqueur de fin de chaîne `\0`.

Exemple:

```
#include <stdio.h>
#include <string.h>

int main(){
    char chaine[] = "j'écrit un truc pour le kiff";
    // Utilisation de strlen
    int taille = strlen(chaine);

    for(int i=0; i<taille; i++){
        printf("%c", chaine[i]);
    }
    printf("\n");
    return 0;
}
```

Remarque

On peut aussi utiliser `sizeof()` que l'on a déjà eu l'occasion de parler pour avoir la taille mémoire totale du tableau, ce qui inclus le marqueur de fin de chaîne `\0`.

```
#include <stdio.h>

int main(){
    char chaine[] = "j'écrit un truc pour le kiff";
    // Utilisation de sizeof
    int taille = sizeof(chaine);

    for(int i=0; i<taille; i++){
        printf("%c", chaine[i]);
    }
    printf("\n");
    return 0;
}
```

```
j'écrit un truc pour le kiff.
```

Le caractère de marqueur de fin de chaîne `\0` revient en fait à mettre un 0 dans l'emplacement mémoire qui lui est dédié. Et, comme ce n'est pas un caractère imprimable comme `a` par exemple, il peut être représenté par différent symbole, ici pour mon compilateur ce sera `·` *le point médian*.

Ainsi, la différence entre `strlen` et `sizeof` est que l'un d'eux inclus `\0` dans la taille de la chaîne, cela peut donc rendre certains programme faux par exemple pour programmer un pendu, si on utilise `sizeof`, on va devoir trouver un mot à t caractères, alors qu'en vrai, c'est un mot à $t - 1$ caractères. Du coup si vous utilisez `sizeof` il faut penser au -1 .

On peut aussi utiliser une boucle `while` pour afficher la chaîne de caractères, elle s'arrête alors quand on trouve le marqueur de fin `\0`.

Exemple:

```
#include <stdio.h>

int main(){
    char chaine[] = "j'écrit un truc pour le kiff";

    int i=0;
    while(chaine[i]!='\0'){
        printf("%c", chaine[i]);
        i++;
    }

    printf("\n");
    return 0;
}
```

```
j'écrit un truc pour le kiff
```

Remarque

On peut utiliser `putchar(char)` à la place du `printf` pour afficher un caractère. la fonction est valide avec une boucle `while` et avec une boucle `for` aussi.

```
#include <stdio.h>

int main(){
    char chaine[] = "j'écrit un truc pour le kiff";

    int i=0;
    while(chaine[i]!='\0'){
        putchar(chaine[i]);
        i++;
    }

    printf("\n");
    return 0;
}
```

j'écrit un truc pour le kiff

Warning

Il faut bien faire attention, un tableau de caractères et une chaînes de caractères ne représentent pas la même chose.

```
char chaine[12] = {'i', 'n', 'f', 'o', 'r', 'm', 'a', ...}
```

est un tableau de 12 caractères sans marqueur de fin de chaînes `\0`, on ne peut pas utiliser le format `%s` pour l'afficher avec `printf` car dans ce cas, `%s` attend une chaîne avec `\0`, ce qui n'est pas le cas ici.

Caractéristique	Tableau de caractères	Chaîne de caractères
Terminaison	Pas forcément <code>'\0'</code>	Toujours <code>'\0'</code>
Taille	Fixe (déclarée)	Fixe ou déduite automatiquement avec littéral
Compatible avec <code>%s</code>	Non	Oui

Lecture

La fonction `scanf`

Présentation et utilisation

La fonction `scanf` permet de lire des saisies de données que se soit des lettres, chiffres, ou alors des chaînes de caractères entières.

Malgré tout, elle possède quelques inconvénient à ne pas négliger pour ne pas être surpris.

Pour bien commencer, étudions le prototype de la fonction `scanf`. Pour rappel, le prototype d'une fonction consiste juste à donner le type de retour de la fonction, son nom et ses arguments.

```
int scanf(const char* format, ...);
```

Oui, le prototype de ma fonction `scanf` est un peu perturbant, il prend au moins 1 paramètre. Les `...` signifient qu'il peut en prendre un nombre variable. On appelle cela une **fonction variadique**, c'est lorsqu'une fonction peut prendre un nombre variable de paramètres. Comme la fameuse fonction `printf` qui prend en paramètre au moins une chaîne de caractères puis ensuite 0 ou *n* paramètres supplémentaires correspondant aux données à afficher si il y en a.

Pour en revenir au prototype de `scanf`,

- Le paramètre `format` désigne simplement le type des variables à saisir, par exemple `%d` pour un entier.
- La fonction renvoie un entier `int`, c'est le nombre de variables affectées par la saisie, ça permet en fait de vérifier si la saisie s'est déroulée correctement.
- En cas d'erreur la fonction retourne `E0F`. *explications plus bas* [^E0F](#)

Exemple :

Un exemple assez basique pour l'utilisation de `scanf`. On demande à l'utilisateur de saisir son âge puis on l'affiche.

```
#include <stdio.h>

int main() {
    int age;
    printf("Entrez votre age : ");
    scanf("%d", &age);
    printf("\nVous avez %d ans.", age);
    return 0;
}
```

Entrez votre age : 20
Vous avez 20 ans.

Il y a quelques petites choses à voir sur ce code.

- Dans l'utilisation de `scanf` pourquoi on met `&age` pourquoi il y a un `&` ?
- Comment on peut savoir si la donnée que l'utilisateur entre est valide ?

Pour comprendre pourquoi il y a un `&` dans l'utilisation de `scanf` il suffit de comprendre ce que fait réellement la fonction.

Quand on utilise la fonction `scanf` c'est pour stocker des saisies utilisateurs dans une variable et l'utiliser plus tard. Ainsi, lorsqu'on utilise `scanf` (on va prendre notre exemple comme référence) pour afficher l'âge de l'utilisateur, lors de la saisie utilisateur, après validation, la fonction `scanf` recopie la saisie dans l'adresse mémoire de la variable `age` qui doit être spécifiée dans la fonction.

C'est pour cette raison le `&`, en fait quand on utilise `scanf("%d", &age)` on souhaite recopier la saisie utilisateur dans l'adresse mémoire créée pour la variable âge. Pour ce faire, on ajoute `&` avant âge.

Maintenant comment savoir si les données saisies par l'utilisateur sont correctes ?

Regardons ce qu'il se passe si à la place de taper un entier, on tape une chaîne de caractères. *On reprend le même code que l'exemple précédent*

```
Entrez votre age : mot
Vous avez 31415 ans.
```

Alors là c'est étonnant, mais qu'est-ce que c'est ?

En fait on a rentré une chaîne de caractères au lieu d'un entier comme spécifié avec le format `%d` alors la fonction `scanf` échoue et ne modifie pas la variable âge. La valeur de cette dernière reste indéfinie *l'entier 31415 est aléatoire, lors d'une autre exécution, un autre nombre sera affiché c'est pour cette raison que l'on parle de valeur indéfinie.*

Il y a déjà quelques paragraphes, j'expliquais que la fonction `scanf` renvoie un entier, le nombre de variables affectées par la saisie. Et bien voilà comment vérifier la saisie utilisateur. Dans notre exemple on vas vérifier si une variable exactement a été affectée par la saisie si c'est le cas on renvoie l'âge sinon on renvoie un message d'erreur.

Exemple:

```
#include <stdio.h>

int main() {
    int age;
    int result; // Variable pour stockée le résultat de scanf
    printf("Entrez votre age : ");
    result = scanf("%d", &age);
    if(result==1)
        printf("Saisie valide, Vous avez %d ans.", age);
    else
        printf("\nErreur de saisie.\n");
    return 0;
}
```

```
Entrez votre age : 20
Saisie valide, Vous avez 20 ans.
```

```
Entrez votre age : mot
Erreur de saisie.
```

Aussi, on a dit qu'en cas d'erreur, la fonction `scanf` renvoie `E0F`. Cela mérite quelques explications.

Pour clarifier un peu,

la fonction `scanf` ne renvoie pas toujours `E0F` *End Of File*, c'est seulement si une erreur d'entrée/sortie se produit ou alors si on atteint la fin d'un fichier, par exemple `Ctrl+Z` sur Windows. *Notion expliquée lors du cours sur les fichiers.* Si la saisie utilisateur ne correspond pas au format attendu, la fonction `scanf` renverra 0.

Utilisation avancée

On peut aussi lire plusieurs données en même temps avec la fonction `scanf`, c'est pour cette raison qu'elle possède un nombre de paramètres variables.

Exemple:

On souhaite lire un mot puis un nombre.

```
#include <stdio.h>

int main() {
    char prenom[100];
    int age;

    printf("Entrez votre prénom et votre âge : ");
    scanf("%s%d", prenom, &age);
    printf("Salut %s, tu as %d ans", prenom, age);
    return 0;
}
```

```
Entrez votre prénom et votre âge : Paul 45
Salut Paul, tu as 45 ans
```

Limites

Lecture limitée

Exemple:

On souhaite que l'utilisateur entre son nom et son prénom pour l'afficher.

```
#include <stdio.h>

int main() {
    char identite[100];
    printf("Entrez votre prénom et nom : ");
    scanf("%s", identite);
    printf("\nSalut tu t'appelles : %s", identite);
    return 0;
}
```

```
Entrez votre prénom et nom : Paul Durant
Salut tu t'appelles : Paul
```

Bon je pense que tu as quelques questions à poser :

- Pourquoi dans l'utilisation de `scanf`, cette fois on met pas le `&` de l'adresse mémoire devant identité ?
- Pourquoi le résultat de la saisie est simplement `Paul` et pas `Paul Durant` comme tapé au dessus ?

On y vient, un peu de patience !

En fait, dans notre cas on souhaite passer une chaîne de caractères à notre fonction `scanf`, et on a expliqué qu'en fin de compte c'est un tableau de `char`. Et en langage C, lorsque l'on passe un tableau en paramètre d'une fonction, ce dernier est automatiquement converti en pointeur vers le premier élément de ce dernier.

Pour simplifier, quand on passe un tableau en paramètre de fonction c'est la même chose que si on passait l'adresse mémoire du premier élément du tableau. *La notion de pointeur arrive...* D'où le fait qu'ici on a pas besoin de `&`.

Un second problème se dresse devant nous, on a tapé `Paul Durant` et le programme a affiché `Paul` uniquement. Que s'est-il passé pour `Durant` ?

La fonction `scanf` s'arrête de lire une saisie utilisateur si elle tombe sur un espace, une tabulation ou si on appuie sur entrée. Alors le second mot, ici `Durant` n'est donc pas récupéré.

Alors il n'est pas désintégré, au contraire même il est toujours en mémoire dans ce qu'on appelle un **buffer**, c'est-à-dire qui signifie que la prochaine fois que l'on appellera la fonction `scanf`, elle lira toute seule `Durant` qui est "resté en plan" dans la mémoire.

Observons à l'aide d'un exemple

Exemple :

On reprend le même que précédemment et on utilise 2 fois `scanf`

```
#include <stdio.h>

int main() {
    char identite[100];
    printf("Entrez votre prénom et nom : ");
    scanf("%s", identite);
    printf("\nSalut tu t'appelle : %s", identite);

    // Seconde utilisation
    printf("\nEntrez votre nouveau prénom et nom : ");
    scanf("%s", &identite);
    printf("\nSalut maintenant tu est : %s\n", identite);
    return 0;
}
```

```
Entrez votre prénom et nom : Paul Durant
Salut tu t'appelle : Paul
```

```
Entrez votre nouveau prénom et nom :
Salut maintenant tu est : Durant
```

On remarque bien que lors de la seconde utilisation de `scanf`, l'utilisateur ne saisit rien, c'est la fonction elle-même qui va lire le mot laissé dans la mémoire puis l'affiche.

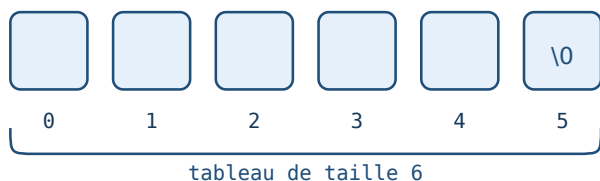
Dépassement de mémoire

Le dépassement de mémoire est un vrai cauchemar pour tout développeur, pour comprendre ce concept plus que fondamental, nous allons l'expliquer avec un exemple et des schémas visuels semblables aux précédents.

Création d'une variable `domaine` d'une taille définie, ici 6.

```
char domaine[6];
```

Cela signifie alors que l'on peut stocker un prénom de 5 lettres, car le dernier emplacement mémoire de la chaîne est réservé au marqueur de fin de chaîne `\0` à ne pas oublier !.



Considérons maintenant le code suivant avec l'exécution associée.

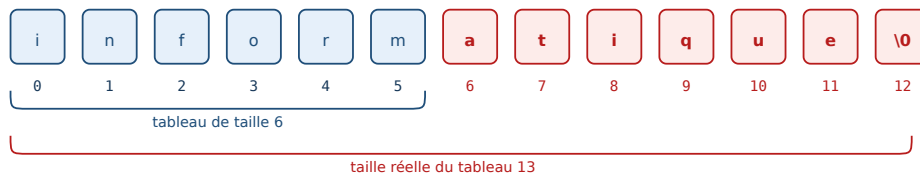
```
#include <stdio.h>

int main() {
    char domaine[6];

    printf("Entrez votre domaine d'étude : ");
    scanf("%s", domaine);
    printf("Domaine : %s", domaine);
    return 0;
}
```

```
Entrez votre domaine d'étude : informatique
Domaine : informatique
```

À première vue, rien ne semble s'être produit. Malheureusement si, on vient de faire ce qu'on appelle un *buffer overflow* ou dépassement de mémoire en français. Ce que nous avons fais c'était créer un tableau pouvant stocker des mots allant jusqu'à 5 lettres. Or ici, on lit le mot `informatique` qui contient 12 lettre. De manière visuelle, on obtient ceci :



En gros le tableau de base est trop petit pour stocker le mot informatique ainsi on le voit en rouge, le dépassement de mémoire.

Cela peut être dangereux car si il n'est pas contrôlé, l'utilisateur peut écrire ce qu'il veut dans la mémoire. Il peut surtout insérer du code en mémoire et faire en sorte qu'il soit exécuté par le programme. C'est l'attaque par *buffer overflow*, une attaque pirate très difficile à faire.

Pour plus d'informations sur le dépassement et ses dangers : https://fr.wikipedia.org/wiki/D%C3%A9passement_de_tampon

La fonction `gets`

La fonction `gets` permet de lire une ligne entière de l'entrée standard jusqu'au retour à la ligne `\n` et de stocker cette dernière dans un tableau fourni.

Exemple :

```
#include <stdio.h>

int main() {
    char line[100];

    printf("Entrez une chaîne de caractères : ");
    gets(line);
    printf("Vous avez entré : %s\n", line); // Affichage

    return 0;
}
```

Entrez une chaîne de caractères : une chaîne simple
Vous avez entré : Une chaîne simple

Inutile de passer encore énormément de temps sur `gets`, elle pose les mêmes problèmes que `scanf` : le dépassement de mémoire.

Remarque

Depuis la norme C11, `gets` a été supprimée de la norme C.

La fonction `fgets`

La fonction `fgets` permet de récupérer une chaîne de caractères à partir d'un flux (le plus souvent, `stdin`).

Prototype de la fonction

```
char *fgets(char *var, int t, FILE *stream);
```

- `var` la variable qui va permettre de stocker la chaîne de caractères lue.
- `t` la taille de la chaîne à stocker.
- `stream` le flux sur lequel on récupère la chaîne (`stdin`, fichier, ...)

Il faut savoir que la fonction `fgets` arrête la lecture lorsqu'elle rencontre `\n`, la fin du flux, ou alors après `t-1` caractères lus (`\0` le dernier caractère).

Warning

La fonction `fgets` inclus `\n` dans la chaîne lue si celle ci est rencontrée avant la limite.

Exemple :

```
#include <stdio.h>

int main() {
    char buffer[100];

    printf("Entrez une chaîne de caractères : ");
    fgets(buffer, sizeof(buffer), stdin); // Lecture sécurisée de la chaîne
    printf("Vous avez entré : %s\n", buffer);

    return 0;
}
```

Entrez une chaîne de caractères : Apprendre la prog C
Vous avez entré : Apprendre la prog C

Critère	<code>scanf</code>	<code>gets</code>	<code>fgets</code>
Usage principal	Lire un mot (jusqu'au premier espace)	Lire une ligne complète	Lire une ligne complète ou partie
Limitation de lecture	Ne lit qu'un mot, pas les espaces	Aucun contrôle sur la taille	Limité à <code>n-1</code> caractères
Sécurité	Peu sûr : peut dépasser le buffer si l'utilisateur tape trop	Très dangereux : risque de débordement mémoire	Sûr si <code>n</code> correct
Gestion du <code>\n</code>	Le <code>\n</code> reste dans le buffer	Supprime le <code>\n</code> automatiquement	Conserve le <code>\n</code> si présent (à retirer manuellement si nécessaire)
Remarques	Utiliser avec spécificateur de longueur : <code>%99s</code> pour un buffer de 100	Obsolète et déconseillé, supprimé depuis C11	Recommandé pour la lecture de chaînes. Supprimer le <code>\n</code> et vider le flux si la ligne est trop longue

Fonctions de manipulation

Copier

En programmation C, on peut avoir besoin de copier une chaîne de caractère et il existe 2 méthodes possibles.

Parcourir et recopier

Ce qui semble le plus intuitif pour commencer c'est de parcourir la chaîne à copier et copier chaque élément un à un dans une seconde chaîne.

Exemple :

```
#include <stdio.h>

int main() {
    char chaine[] = "Bonjour le monde";
    char copie[100];

    int i = 0;
    while(chaine[i]!='\0') {
        copie[i]=chaine[i];
        i++;
    }
    chaine[i]='\0';
    printf("Chaîne : %s \n|| Copie : %s", chaine, copie);
    return 0;
}
```

```
Chaîne : Bonjour le monde
|| Copie : Bonjour le monde
```

Warning

Ne pas oublier d'ajouter le `(\0)` à la fin de la chaîne.

La fonction `strcpy`

La fonction `strcpy` de la bibliothèque `<string.h>` permet de copier une chaîne `origine` (`\0` inclus) dans une chaîne `dest`.

```
char* strcpy(char *dest, const char *origine)
```

- elle renvoie un pointeur vers la chaîne copiée.

```
#include <stdio.h>
#include <string.h>

int main() {
    char chaine[] = "Bonjour le monde";
    char copie[100];

    strcpy(copie, chaine);
    printf("Chaîne : %s \n|| Copie : %s", chaine, copie);
    return 0;
}
```

```
Chaîne : Bonjour le monde
|| Copie : Bonjour le monde
```

Concaténer

Définition

On considère deux chaînes de caractères `ch1` et `ch2` le terme **concaténer** revient à assembler les deux chaînes l'une après l'autre.

Fonction `strcat`

```
char *strcat(char *destination, const char *origine);
```

- On ajoute `origine` à la fin de `destination`
- La fonction renvoie un pointeur vers le premier caractère de `destination`

Il faut faire attention, la fonction `strcat` ne vérifie pas si la taille du tableau `destination` est assez grande pour ajouter `origine` à la fin, pensez à trouver une alternative !. Buffer overflow !

```
#include <stdio.h>
#include <string.h>

int main() {
    char ch[50] = "Bonjour, ";
    char ajout[] = "monde !";

    strcat(ch, ajout); // ajoute ajout à la fin de ch
    printf("%s\n", ch);

    return 0;
}
```

Bonjour, monde !

On peut réfléchir à une solution pour concaténer les chaînes de caractères sans avoir de dépassement de mémoire. *La solution suivante vient d'une réflexion personnelle ce n'est pas forcément la meilleure chose à faire...*

Si on considère ch_1 et ch_2 deux chaînes de tailles t_1 et t_2 respectivement (sans avoir de tableau trop grand) donnée par la fonction `strlen()`. Alors La taille du tableau $ch_1 + ch_2$ est donné par $(t_1 + t_2) + 1$ en comptant le caractère nul.

Exemple:

```
#include <stdio.h>
#include <string.h>

int main() {
    char ch[] = "Bonjour ";
    char ajout[] = "tout le monde !";

    int taille_ch = strlen(ch);
    int taille_ajout = strlen(ajout);
    int taille_totale = taille_ch + taille_ajout + 1; // +1 pour le caractère nul

    char resultat[taille_totale];
    strcpy(resultat, ch); // copie ch dans resultat
    strcat(resultat, ajout); // ajoute ajout à la fin de resultat

    printf("%s\n", resultat);

    return 0;
}
```

Bonjour tout le monde !

Voici l'algorithme de la solution proposée pour éviter les dépassements mémoire.

```
concaténation
Définition et initialisation implicite
Récupérer les tailles respectives
Calcul taille totale

Définition d'un tableau de résultat de taille_totale
On copie origine dans resultat
On concatène ajout dans resultat

On affiche résultat
fin
```

Bon ok, je sais c'est un peu plus long mais bon là pas de dépassement mémoire.

Fonction `strncat`

En langage C, la fonction `strncat` (définie dans `<string.h>`) permet de concaténer (ajouter) une chaîne de caractères à la fin d'une autre, en limitant le nombre de caractères copiés.

```
char *strncat(char *destination, const char *origine, size_t n);
```

- `destination` : la chaîne de destination (doit être assez grande pour contenir le résultat).
- `origine` : la chaîne à ajouter.
- `n` : le nombre maximum de caractères à copier depuis `origine`.
Elle renvoie aussi un pointeur vers `destination`.

Warning

- Il faut que le tableau `destination` soit assez grand pour contenir la chaîne finale + `\0` (le caractère de fin de chaîne).
- `strncat` ajoute toujours `\0` à la fin.

Exemple:

```
#include <stdio.h>
#include <string.h>

int main() {
    char dest[20] = "Bonjour";
    char src[] = "_le monde !";

    // On ajoute seulement les 5 premiers caractères de src
    strncat(dest, src, 5);

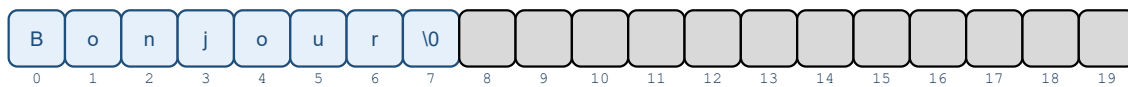
    printf("Résultat : %s\n", dest)
    return 0;
}
```

Bonjour_le m

On peut illustrer la concaténation de l'exemple précédant de la manière suivante :

AVANT

dest[20] = "Bonjour"

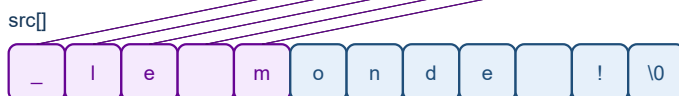
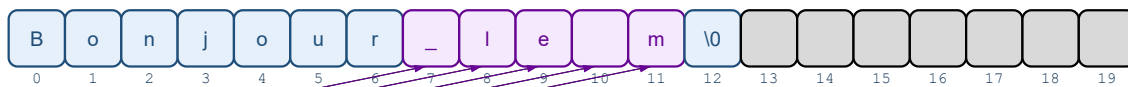


src[] = "_le monde !"



APRÈS

dest[20] = "Bonjour_le m"



5 caractères copiés depuis src vers dest (indices 7 à 11)

Comparer

Rechercher

Découper